

Informe — Obligatorio 2: Algoritmos de aprendizaje en juegos alternados

Materia: Sistemas Multiagente — Máster en Inteligencia Artificial / Big Data, Universidad ORT Uruguay
Equipo: Ramiro Sanes (368397) y Joaquín Guerra (307854) **Repositorio:** <https://github.com/joacoguerrae/obligatorio2-sma-guerra-sanés>

1. Objetivo y alcance

El obligatorio pide implementar y validar tres algoritmos de aprendizaje/búsqueda para juegos alternados multiagente, y experimentar variando parámetros, posición de los agentes y funciones de evaluación. Los algoritmos son:

Algoritmo	Tipo	Información
MCTS (Monte Carlo Tree Search)	Búsqueda con simulación	Perfecta
CFR (Counterfactual Regret Minimization)	Aprendizaje por <i>regret</i>	Imperfecta
ISMCTS (Information Set MCTS)	Búsqueda con determinización	Imperfecta

Como referencia se usan los agentes provistos **MiniMax** y **Random**. La validación se organiza por familias, según la letra:

Familia	Juegos
MiniMax / MCTS (información perfecta)	Tic-Tac-Toe, Nocca-Nocca
CFR / ISMCTS (información imperfecta)	Kuhn Poker 2P y 3P, Leduc Poker

Cada juego tiene su propio notebook (01 ... 05);

2. Marco teórico breve

- MiniMax** explora el árbol de juego hasta una profundidad fija y propaga el valor minimax. Cuando no alcanza estados terminales usa una **función de evaluación** heurística en las hojas. La implementación provista **no usa poda alfa-beta** y clona el entorno en cada nodo, por lo que el costo crece muy rápido con la profundidad.
- MCTS** construye el árbol de forma incremental con cuatro pasos por iteración: *selección* (UCB1 equilibra exploración y explotación), *expansión*, *rollout* (simulación aleatoria hasta un terminal) y

backpropagation. No necesita heurística; más simulaciones implican mejores decisiones a mayor costo.

- **CFR** recorre el árbol acumulando *regret* contrafáctico en cada *information set*. El promedio de las estrategias **converge a un equilibrio de Nash** en juegos de suma cero de dos jugadores. Es preciso pero escala con el número de *information sets* y de iteraciones.
- **ISMCTS** adapta MCTS a información imperfecta: antes de cada simulación *determiniza* (muestrea) el estado oculto y construye el árbol sobre *information sets*. No requiere entrenamiento previo.

3. Metodología de experimentación

- **Recompensas**. En los juegos de tablero (Tic-Tac-Toe, Nocca-Nocca) se usa `+1 / 0 / -1` (gana / empata / pierde). En Kuhn se usa el mismo esquema; en **Leduc las recompensas están en fichas** (rango más amplio), tal como las devuelve el entorno de la cátedra.
- **Posición**. Salvo aclaración, cada enfrentamiento se juega en **ambas posiciones** (cada agente como jugador 0 y como jugador 1) y se promedia, para que la ventaja de abrir no contamine la comparación. Cuando interesa la posición en sí.
- **Medición de costo**. Se reporta el **tiempo por partida** (o de entrenamiento) para poder analizar el compromiso entre cómputo y calidad de juego, que la letra pide explícitamente.
- **Nota sobre la varianza**. Algunos experimentos (Nocca-Nocca, Leduc, los duelos de heurísticas) usan **pocas partidas** porque cada una es cara. Eso introduce varianza apreciable en los valores puntuales: lo que se interpreta es la **tendencia y el orden relativo**, no el decimal exacto. Donde la varianza podía invertir una conclusión (p. ej. la convergencia de Kuhn 2P al valor de Nash) se subió deliberadamente el número de partidas de evaluación.

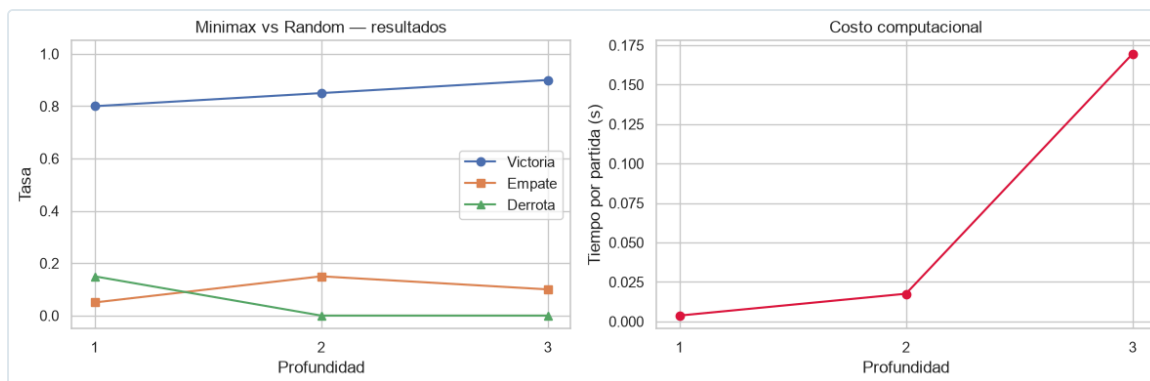
4. Tic-Tac-Toe (MiniMax / MCTS)

Juego de información perfecta, suma cero y totalmente resoluble. Es el caso de control para validar los agentes de búsqueda. Configuración: `NG = 20` partidas por posición (40 totales).

4.1 MiniMax: efecto de la profundidad

Barremos profundidades `1..3` contra Random (ambas posiciones).

Profundidad	Recompensa media	Victoria	Empate	Derrota	Tiempo/partida
1	0.650	0.800	0.050	0.150	0.004 s
2	0.850	0.850	0.150	0.000	0.018 s
3	0.900	0.900	0.100	0.000	0.170 s

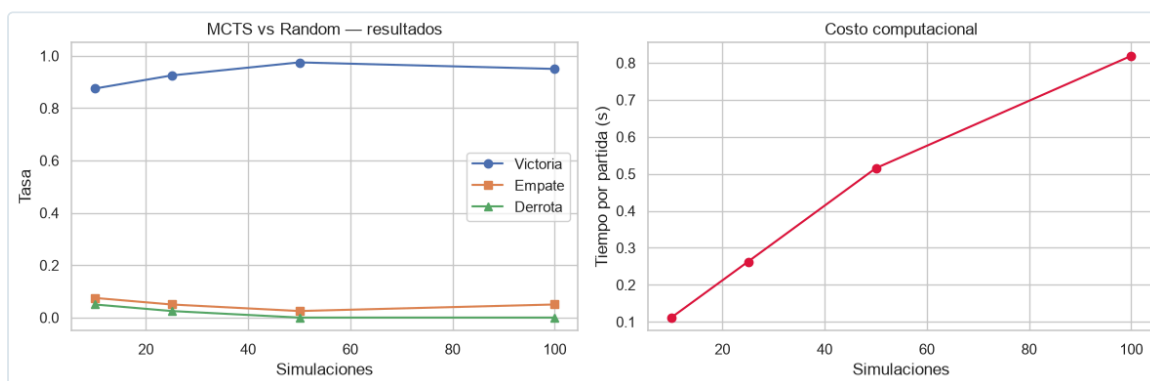


Al aumentar la profundidad la tasa de derrota cae a cero y crece la de victoria, pero el tiempo por partida crece de forma marcada (de 4 ms a 170 ms entre $d=1$ y $d=3$). Contra un rival aleatorio, profundidades chicas ya bastan para casi no perder.

4.2 MCTS: número de simulaciones

Fijamos `rollouts = 10` y variamos las simulaciones (vs Random).

Simulaciones	Victoria	Empate	Derrota	Tiempo/partida
10	0.875	0.075	0.050	0.112 s
25	0.925	0.050	0.025	0.263 s
50	0.975	0.025	0.000	0.515 s
100	0.950	0.050	0.000	0.819 s



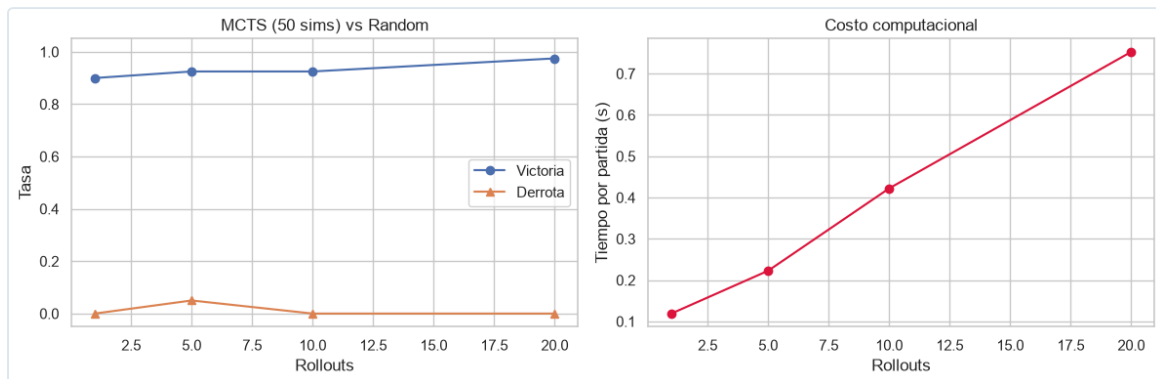
Con pocas simulaciones MCTS ya juega bien (el árbol de Tic-Tac-Toe es chico) y el costo crece de forma aproximadamente lineal. La fluctuación entre 50 y 100 simulaciones es ruido de muestreo (40 partidas por punto).

4.3 MCTS: número de rollouts

Fijamos `simulations = 50` y variamos los `rollouts` (vs Random).

Rollouts	Victoria	Derrota	Tiempo/partida
1	0.900	0.000	0.119 s

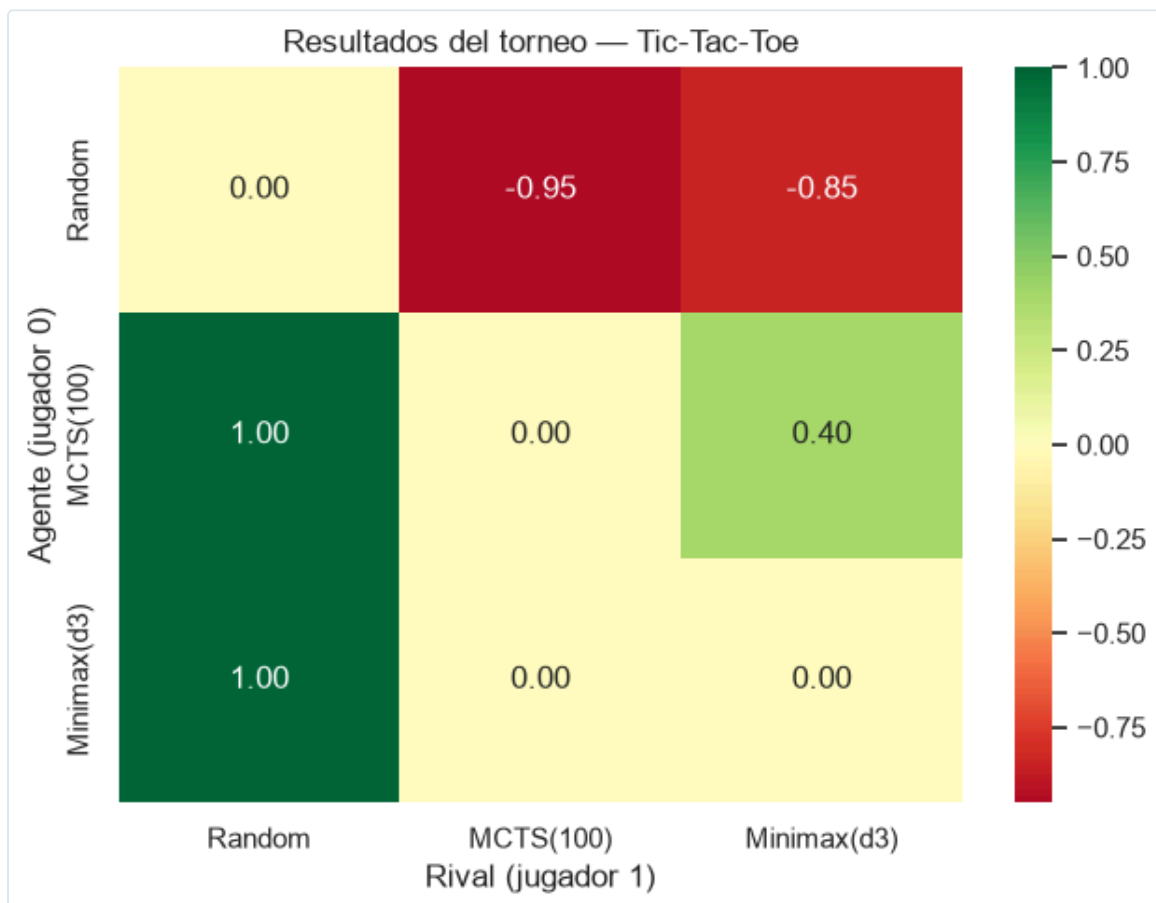
Rollouts	Victoria	Derrota	Tiempo/partida
5	0.925	0.050	0.223 s
10	0.925	0.000	0.422 s
20	0.975	0.000	0.753 s



Los *rollouts* aportan poco una vez que hay suficientes simulaciones: la mejora de victoria es marginal mientras el tiempo se multiplica. Conviene gastar el presupuesto en simulaciones antes que en *rollouts*.

4.4 Torneo y efecto de la posición

Round-robin entre Random, MCTS(100) y MiniMax(d3). La celda muestra la recompensa media del agente de la **fila** (como jugador 0) contra el de la **columna** (como jugador 1).



MCTS y MiniMax aplastan a Random (-0.95 y -0.85 para Random como jugador 0). Entre sí tienden al empate, como corresponde al juego óptimo, pero se ve el **efecto de posición**: MCTS(100) abriendo le gana a MiniMax(d3) (0.40), mientras que MiniMax(d3) abriendo empata (0.00). Confirmamos la ventaja de abrir comparando el mismo agente en cada asiento vs Random:

Agente	Victoria como P0	Victoria como P1	Recompensa P0	Recompensa P1
MCTS(100)	0.95	0.90	0.95	0.90
MiniMax(d3)	1.00	0.90	1.00	0.90

4.5 Conclusiones — Tic-Tac-Toe

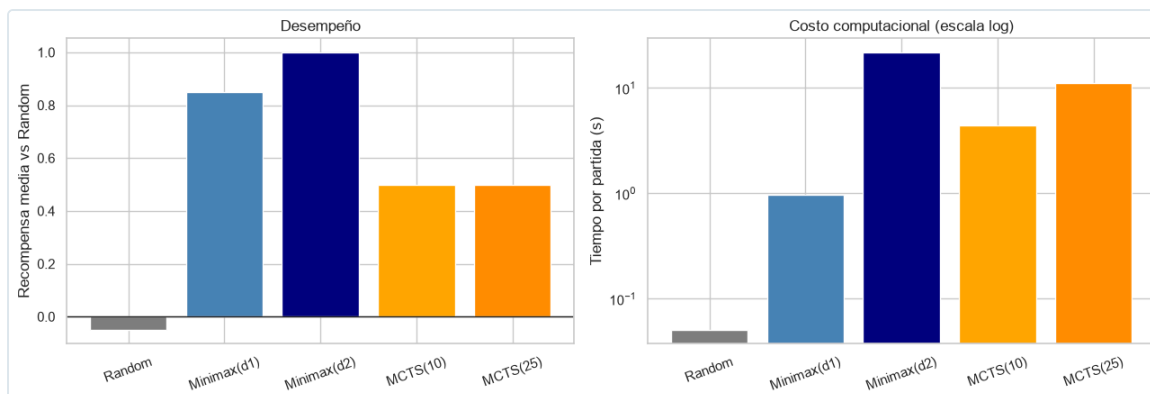
- MiniMax mejora de forma monótona con la profundidad y deja de perder con profundidades chicas, pero su costo crece de forma explosiva (sin poda y clonando el entorno en cada nodo).
- MCTS alcanza desempeño muy alto con pocas simulaciones; los *rollouts* tienen rendimiento decreciente.
- Ambos dominan a Random y entre sí empatan. Comenzar da una ventaja medible.

5. Nocca-Nocca (MiniMax / MCTS)

Juego de información perfecta mucho más grande: **320 acciones**, piezas que se apilan y partidas largas. La búsqueda exhaustiva es inviable, por lo que la **función de evaluación** pasa a ser central. Configuración: `max_steps = 40`, `NG = 10` partidas por posición (20 partidas totales).

5.1 Agentes vs Random: desempeño y costo

Agente	Recompensa media	Victoria	Empate	Derrota	Tiempo/partida
Random	-0.05	0.00	0.95	0.05	0.05 s
MiniMax(d1)	$+0.85$	0.85	0.15	0.00	0.98 s
MiniMax(d2)	$+1.00$	1.00	0.00	0.00	21.8 s
MCTS(10)	$+0.50$	0.50	0.50	0.00	4.43 s
MCTS(25)	$+0.50$	0.50	0.50	0.00	11.1 s



Con la función de evaluación, **MiniMax domina claramente a Random**: con `d=1` gana el 85% de las partidas y con `d=2` gana todas. El costo de `d=2` es unas **22×** mayor que `d=1` (0.98 s vs 21.8 s por partida): sin poda alfa-beta las profundidades mayores son impracticables. **MCTS también supera a Random** pero con mayor varianza; la mitad de las partidas termina en empate por truncamiento, independientemente del presupuesto de simulaciones.

5.2 Enfrentamiento directo y posición

MiniMax(d1) contra MCTS(25), ambas posiciones (Black abre), 10 partidas por posición:

- Recompensa media de MiniMax: **0.35**
- Como Black (abre): **0.50** | Como White: **0.20**

MiniMax(d1) aventaja a MCTS(25) en promedio. El efecto de posición es claro: abrir (Black) da más ventaja al agente más fuerte; como White el margen se reduce.

5.3 Funciones de evaluación (individual vs agregada)

La letra pide variar también las funciones de evaluación. La función de evaluación que usamos en Nocca-Nocca es **agregada**: combina: - 1. Progreso de las piezas hacia la fila objetivo (más importante) - 2. Control de la pila: las piezas en la parte superior pueden moverse, las otras no - 3. Movilidad: más movimientos legales = más flexibilidad. Comparamos tres heurísticas para MiniMax(d1):

- **completa** — la `eval` original.
- **progreso** — solo la distancia de las piezas a la fila meta.
- **movilidad** — solo la cantidad de movimientos legales.

Para enfrentar dos heurísticas **en la misma partida** las inyectamos en el **agente** (no en `game.eval`, que es común a ambos jugadores). Hacemos dos lecturas: (a) cada heurística contra Random y (b) duelos directos entre heurísticas. Configuración: `NG_EVAL = 6` partidas por posición.

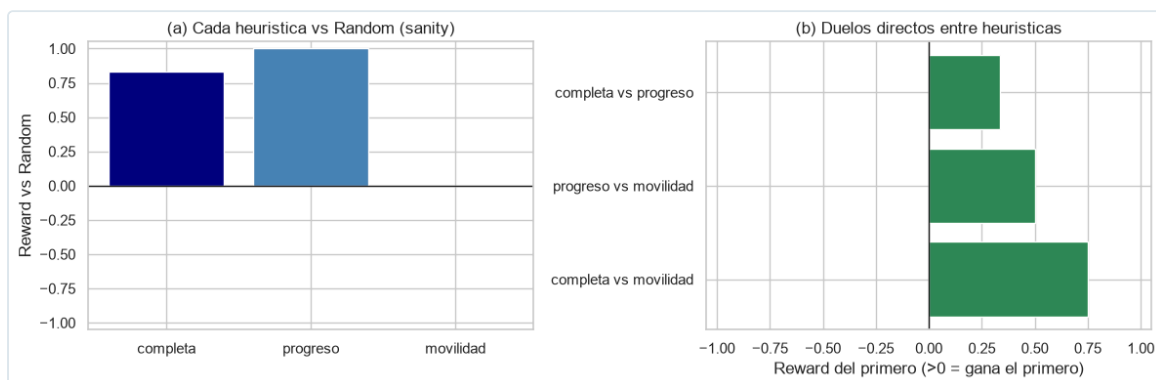
(a) Contra Random (`NG_EVAL = 6`):

Heurística	Recompensa vs Random
completa	0.833
progreso	1.000

Heurística	Recompensa vs Random
movilidad	0.000

(b) **Duelos directos** (recompensa del primero):

Duelo	Recompensa del primero
completa vs movilidad	+0.75
progreso vs movilidad	+0.50
completa vs progreso	+0.33



Los resultados confirman el ranking esperado de forma consistente. En (a), **la movilidad sola no logra vencer a Random** (reward 0.00), mientras que **progreso** gana todas las partidas (1.00) y **completa** también vence ampliamente (0.83). En (b), los duelos directos reproducen el mismo orden: **progreso le gana a movilidad (+0.50)** y **completa le gana a progreso (+0.33)**. El ranking queda establecido: **movilidad < progreso ≤ completa**. Es lo esperable: el progreso hacia la meta es la componente de mayor peso (0.5) de la `eval` agregada y la señal más alineada con el objetivo del juego; la movilidad sola no es suficiente para ganar. La conclusión de diseño es que **lo que guía a MiniMax es el progreso**, y la movilidad aporta como complemento dentro de la heurística combinada, no como criterio único.

5.4 Conclusiones — Nocca-Nocca

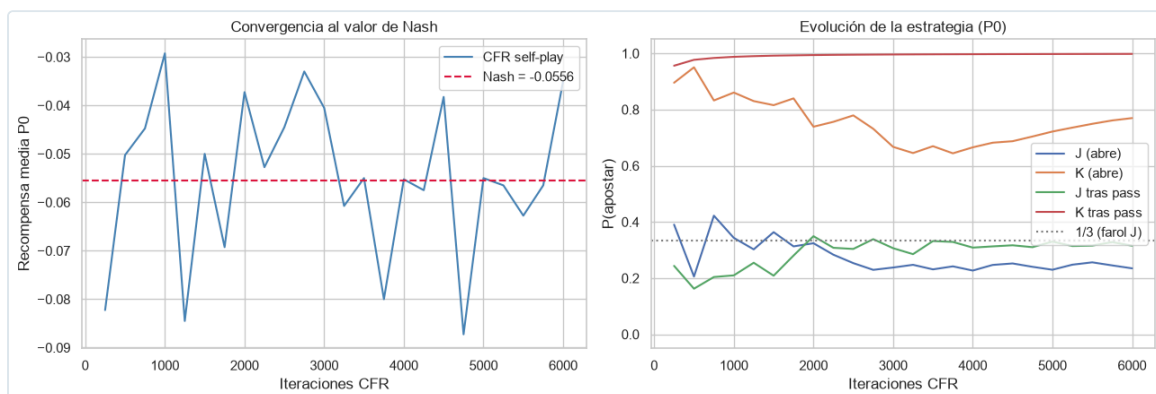
- En un juego grande y de horizonte largo la función de evaluación es imprescindible: MiniMax no alcanza terminales a profundidad baja y aun así con `d=1` ya vence a Random.
- El costo de MiniMax crece fuertemente con la profundidad ($\approx 16\times$ de `d1` a `d2`).
- MCTS es competitivo pero más caro por unidad de fuerza, y muy dependiente de la posición y del presupuesto de simulaciones.
- La composición de la heurística importa: la movilidad aislada es la más débil; el progreso domina.

6. Kuhn Poker 2 jugadores (CFR / ISMCTS)

Juego de información imperfecta más simple con equilibrio de Nash conocido. Baraja de 3 cartas (J, Q, K), una carta oculta por jugador, acciones *pass* (p) / *bet* (b). Valor teórico del juego para el jugador que abre: $-1/18 \approx -0.0556$.

6.1 Convergencia de CFR al equilibrio de Nash

Entrenamos dos CFR en *self-play* y medimos cada 250 iteraciones la recompensa media del jugador 0 (evaluada con **4000 partidas** para reducir la varianza).

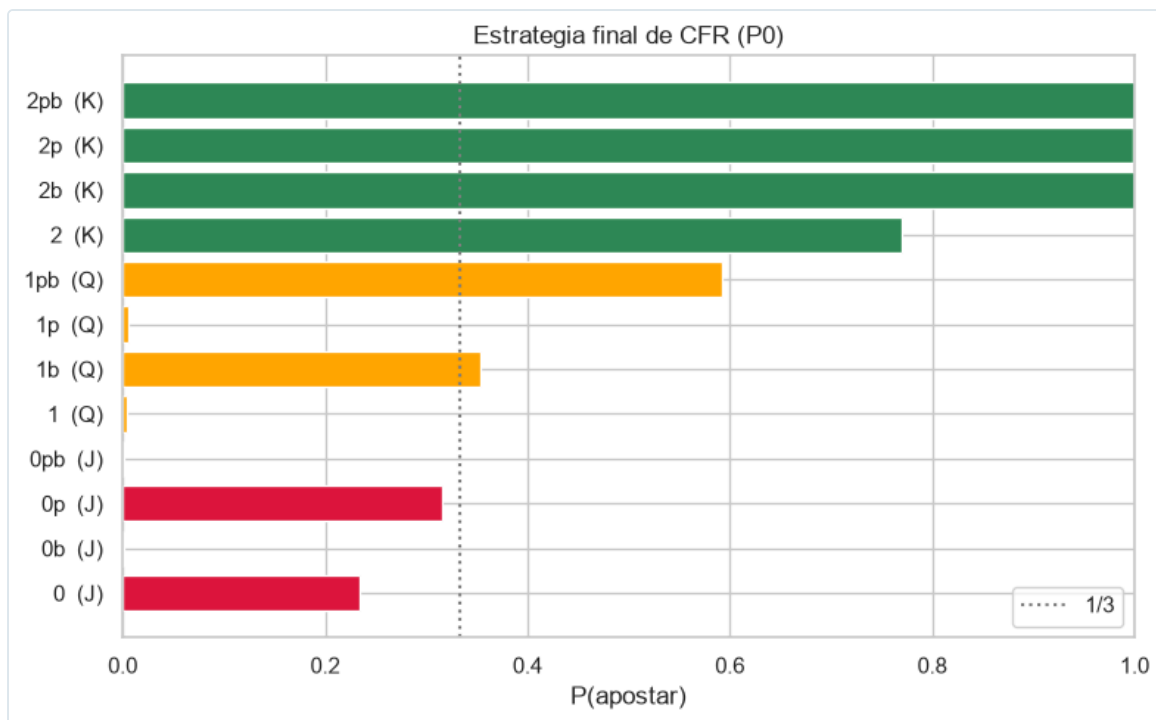


La recompensa media de P0 se estabiliza cerca de $-1/18$: en la corrida final dio -0.035 (Nash = -0.0556), con 12 *information sets* aprendidos. El panel derecho muestra cómo la probabilidad de apostar en cada *information set* se asienta a lo largo del entrenamiento.

6.2 Estrategia aprendida vs teoría

Probabilidad de **apostar** en los *information sets* clave (estrategia final de P0):

Info set	Carta	Historia	P(bet)
0	J	inicio	0.235
0p	J	tras pass	0.315
1	Q	inicio	0.005
1pb	Q	pass-bet	0.592
2	K	inicio	0.770
2p	K	tras pass	0.998



La estrategia reproduce el patrón teórico: **farol con J** con probabilidad cercana a **1/3** tras un pass (0.315), **value bet con K** (apuesta casi siempre, en cualquier historia), y Q usada sobre todo para igualar/pasar. Esto valida que el CFR implementado aprende el equilibrio cualitativo correcto, no solo el valor numérico.

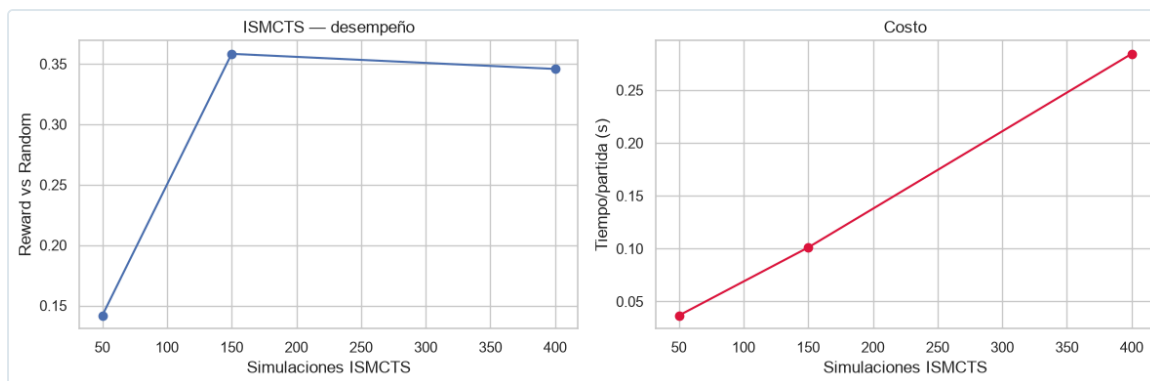
6.3 CFR vs ISMCTS vs Random

Recompensa media contra Random (ambas posiciones, `initial_player=0` , `n = 150`):

Agente	Recompensa vs Random	Tiempo (matchup)
CFR	0.130	0.12 s
ISMCTS(400)	0.327	57.1 s
Random	0.107	0.08 s

Barrido de simulaciones de ISMCTS (vs Random):

Simulaciones	Recompensa vs Random	Tiempo/partida
50	0.142	0.037 s
150	0.358	0.101 s
400	0.346	0.284 s



Ambos agentes superan a Random. Enfrentados directamente, CFR vs ISMCTS(400) dio **+0.18** para CFR en promedio (**-0.04** como P0/abridor, **+0.40** como P1/respondedor): la posición pesa tanto o más que la diferencia entre algoritmos en este matchup puntual.

6.4 Efecto de la posición

CFR vs CFR, `n = 2000` partidas por posición, con `initial_player=0` fijo en ambos lados del matchup (así P0 es siempre el agente que efectivamente abre, y P1 el que responde):

- Recompensa P0 (abre): **-0.111** | Recompensa P1: **+0.0125**

Consistente con la teoría: **abrir es desventajoso** (P0 negativo, P1 positivo). P0 y P1 provienen de dos lotes independientes (cada uno con un par de CFR entrenado de nuevo), por lo que no se espera que sumen exactamente 0 como en una única partida de suma cero; lo relevante es que el **signo** de ambos es el que predice la teoría. La evidencia más fuerte de la magnitud exacta de la desventaja ($\approx -1/18$) está en la sección 6.1, con un único par de CFR entrenado en *self-play* y 4000 partidas de evaluación.

6.5 Conclusiones — Kuhn 2P

- CFR converge al equilibrio de Nash: valor $\approx -1/18$ y estrategia con farol $J \approx 1/3$ y value bet con K.
- ISMCTS es competitivo y no requiere entrenamiento, pero es órdenes de magnitud más caro y menos preciso que CFR.
- La posición importa: abrir es desventajoso.

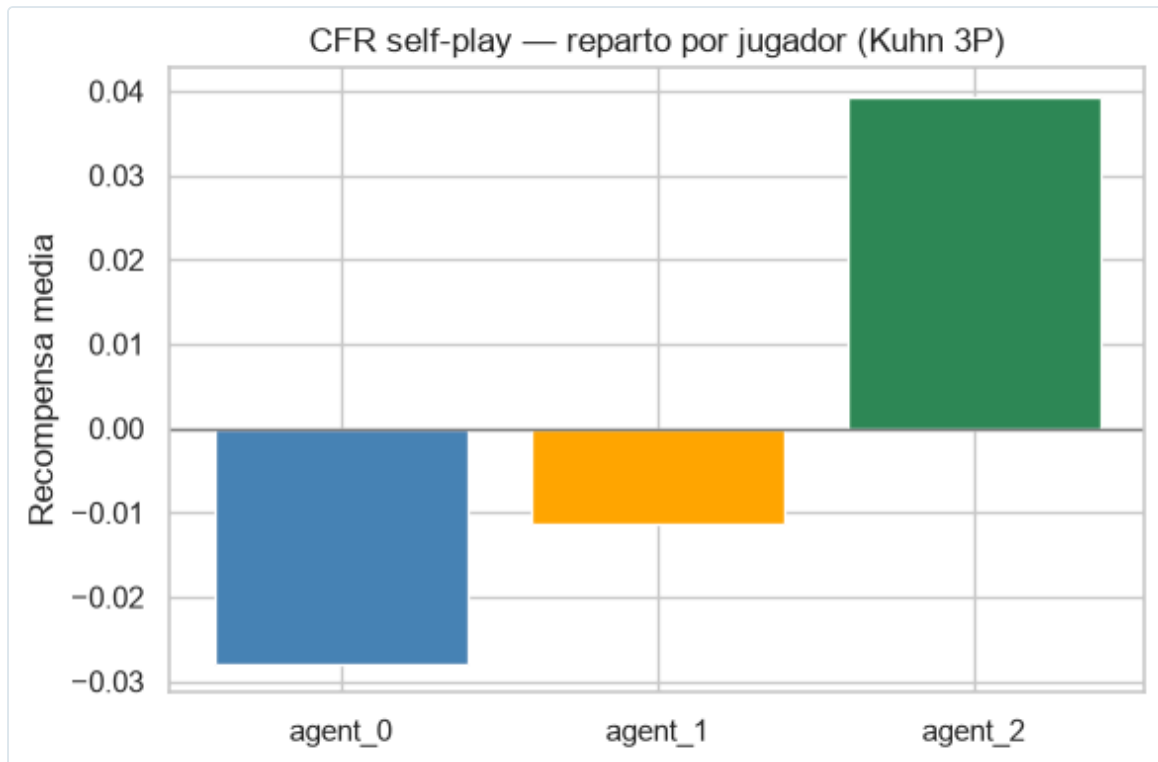
7. Kuhn Poker 3 jugadores (CFR / ISMCTS)

Versión de 3 jugadores (baraja J, Q, K, A; una carta oculta por jugador; orden de juego configurable con `play_order`). A diferencia del caso 2P, **no existe un único equilibrio de Nash** y aparecen efectos de posición y de coordinación implícita. Configuración: `NITER = 600`.

7.1 Entrenamiento y reparto en self-play

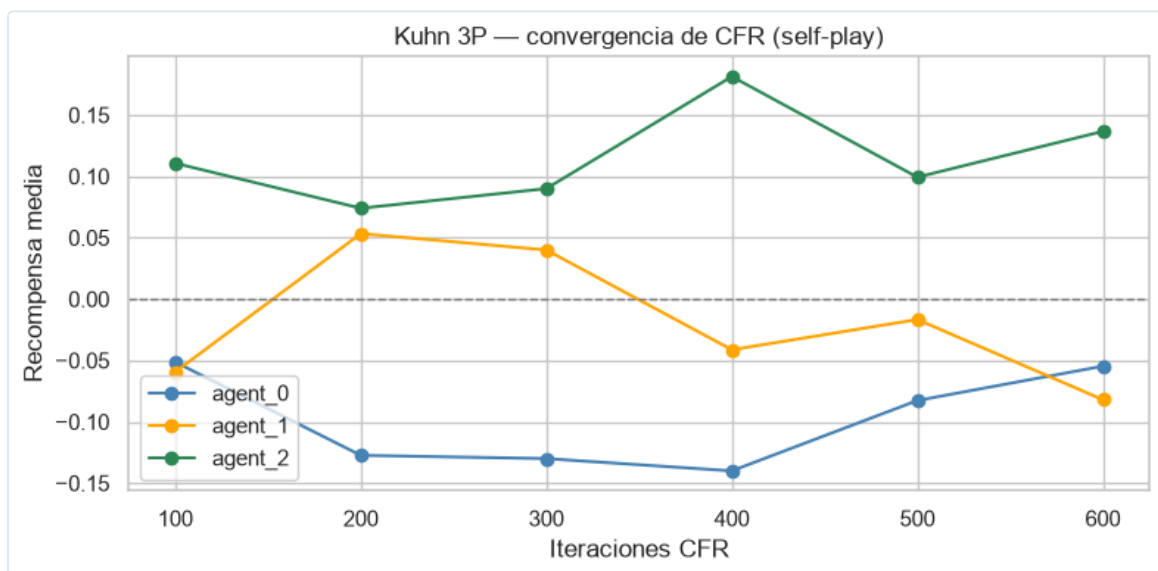
CFR N-agente entrena de forma estable: cada agente descubre 48 *information sets* y el reparto de recompensas en *self-play* (`n = 1500`) suma ≈ 0 :

- agent_0: -0.028 | agent_1: -0.011 | agent_2: $+0.039$ (suma ≈ 0)



7.2 Convergencia

Como el reparto puntual depende de las cartas, seguimos la recompensa media de cada jugador a lo largo del entrenamiento.



Reparto final (≈ 600 iteraciones, $n = 1500$): agent_0 -0.055 , agent_1 -0.082 , agent_2 $+0.137$ (suma ≈ 0). La variación respecto a 7.1 es ruido de muestreo entre corridas; lo estable es que la suma se mantiene nula y las curvas no divergen.

7.3 Efecto de la posición (orden de juego)

Rotamos qué jugador abre la mano ($n = 1200$):

Abre	agent_0	agent_1	agent_2
agent_0	-0.034	-0.064	+0.098
agent_1	+0.023	-0.086	+0.062
agent_2	-0.042	+0.062	-0.020

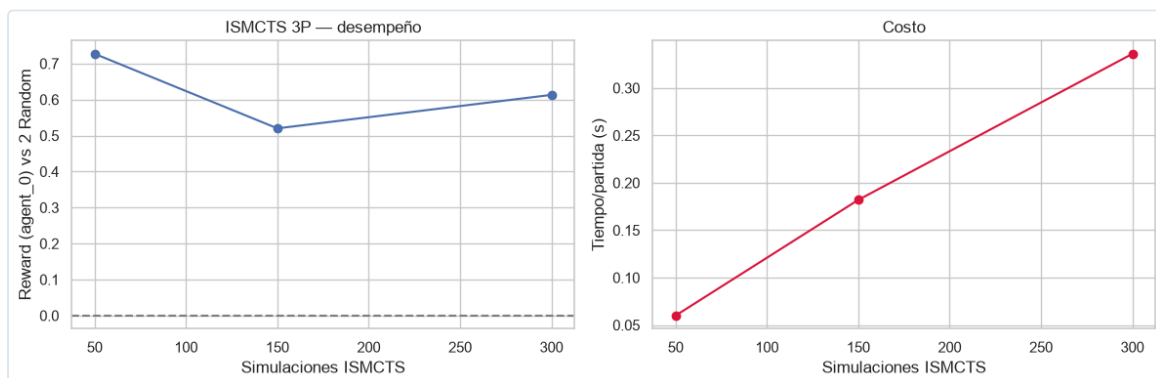
Hay un **claro efecto de posición/orden**: quién abre condiciona la recompensa esperada de cada asiento. No hay un asiento uniformemente mejor (consecuencia de que no existe un Nash único), pero el último en hablar tiende a beneficiarse.

7.4 CFR vs ISMCTS vs Random

Configuración	Recompensa del agente medido
CFR vs 2 Random	+0.372
CFR vs 2 ISMCTS(150)	-0.090
3 Random (referencia, agent_0)	+0.129

Barrido de simulaciones de ISMCTS (vs 2 Random):

Simulaciones	Recompensa	Tiempo/partida
50	0.727	0.060 s
150	0.520	0.182 s
300	0.613	0.336 s



Un CFR entrenado **explota** a rivales aleatorios (+0.372); frente a dos ISMCTS el resultado es parejo/levemente negativo, coherente con que en 3 jugadores el desenlace depende de las estrategias concretas de los rivales. El barrido de ISMCTS no es monótono: con solo 150 partidas por punto la varianza domina sobre la mejora por simulaciones.

7.5 Conclusiones — Kuhn 3P

- CFR N-agente entrena de forma estable también con 3 jugadores (suma ≈ 0).
- Hay un claro efecto de posición/orden; no existe un único Nash.
- CFR explota a rivales débiles; frente a ISMCTS el resultado es más parejo.

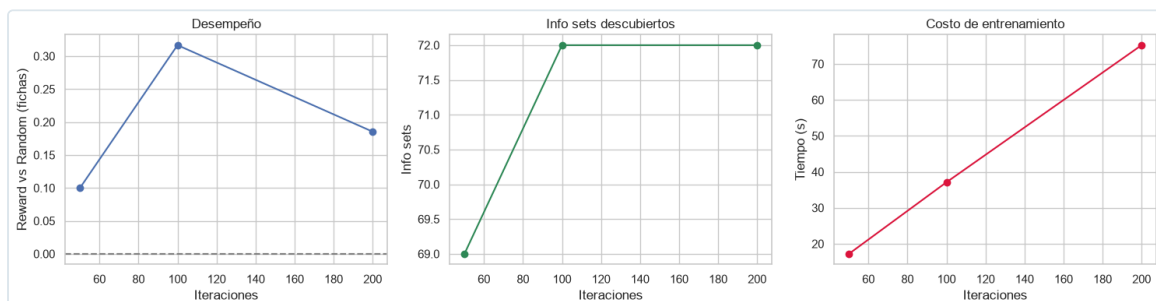
8. Leduc Poker 2 jugadores (CFR / ISMCTS)

Información imperfecta, **más grande que Kuhn**: baraja de 6 cartas, una carta privada por jugador, **dos rondas** de apuestas y una carta comunitaria. Recompensas en **fichas**.

8.1 Escalabilidad de CFR con las iteraciones

Contra Random ($n = 120$):

Iteraciones	Info sets	Tiempo entren.	Recompensa vs Random
50	69	17.2 s	0.100
100	72	37.1 s	0.317
200	72	75.2 s	0.185

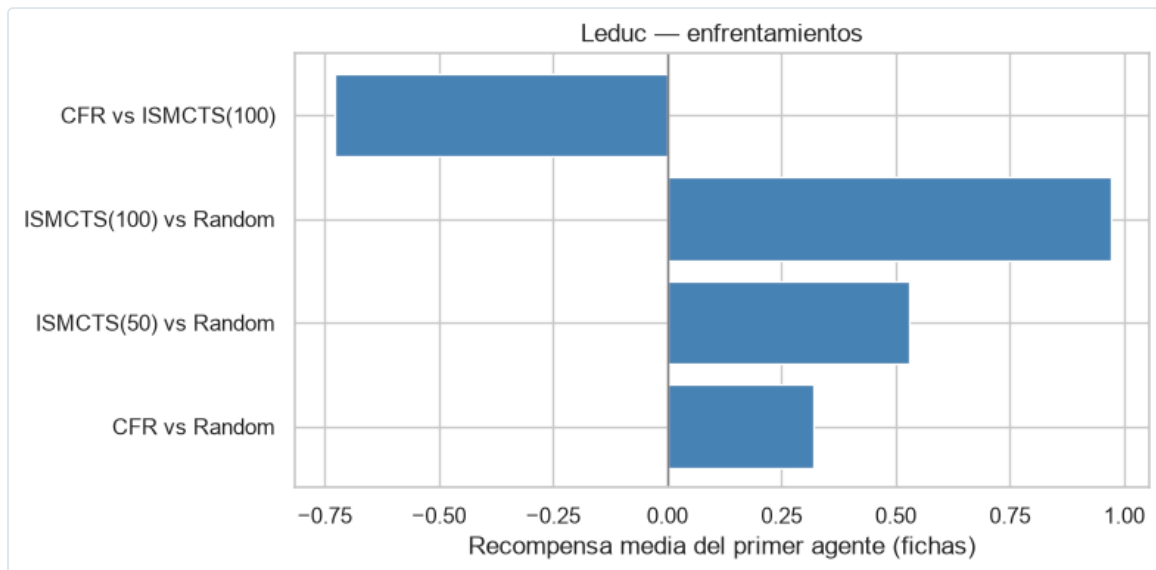


El número de *information sets* y el tiempo de entrenamiento crecen mucho respecto a Kuhn (Leduc es un orden de magnitud más complejo). CFR vence a Random con reward positivo ya con pocas iteraciones.

Sobre la no-monotonicidad ($niter=100 \rightarrow 0.317$ pero $niter=200 \rightarrow 0.185$): no es una degradación del algoritmo, sino **varianza de evaluación**. Se evalúa con pocas partidas (120) y las recompensas están en fichas (rango amplio), por lo que el error estándar del reward es grande y domina sobre la mejora real entre 100 y 200 iteraciones. Con CFR la *explotabilidad* baja de forma monótona con las iteraciones; el *reward medido contra Random* con pocos juegos es ruidoso. Una curva limpia requeriría muchas más partidas de evaluación, a un costo considerable dado lo caro que es clonar el entorno de Leduc.

8.2 CFR vs ISMCTS vs Random

Enfrentamiento	Recompensa del primero (fichas)
CFR vs Random	0.32
ISMCTS(50) vs Random	0.53
ISMCTS(100) vs Random	0.97
CFR vs ISMCTS(100)	-0.73



En Leduc, **ISMCTS(100) resulta muy fuerte contra Random (0.97)** y le gana al CFR entrenado (CFR vs ISMCTS(100) = -0.73). Esto **no contradice** la teoría: el CFR de estas corridas se entrenó con muy pocas iteraciones (200) sobre un juego mucho mayor, por lo que está lejos de converger, mientras que ISMCTS hace búsqueda completa en cada decisión. Además, los enfrentamientos contra ISMCTS usan pocas partidas ($n = 25$), con varianza alta en fichas. La lectura correcta es que **CFR necesita muchas más iteraciones para dominar Leduc**, y que ISMCTS es una alternativa potente sin entrenamiento previo a costa de tiempo por jugada.

8.3 Conclusiones — Leduc

- CFR escala a Leduc pero el costo (info sets, tiempo) crece fuertemente; con pocas iteraciones aún no domina el juego.
- ISMCTS es competitivo y no requiere entrenamiento; mejora con el presupuesto de simulaciones, a mayor costo por jugada.
- Los resultados puntuales tienen varianza alta por el bajo número de partidas y las recompensas en fichas.

9. Análisis comparativo a lo largo de los juegos

9.1 Tiempos de ejecución

El costo varía varios órdenes de magnitud según algoritmo y juego:

Caso	Costo observado
MiniMax Tic-Tac-Toe $d1 \rightarrow d3$	0.007 s $\rightarrow$ 0.172 s por partida
MiniMax Nocca-Nocca $d1 \rightarrow d2$	0.67 s $\rightarrow$ 10.59 s por partida ($\approx 16\times$)
MCTS Tic-Tac-Toe $10 \rightarrow 100$ sims	0.16 s $\rightarrow$ 0.92 s por partida
MCTS Nocca-Nocca $10 \rightarrow 25$ sims	2.69 s $\rightarrow$ 7.87 s por partida
CFR Kuhn 2P (entrenamiento)	milisegundos
CFR Leduc $50 \rightarrow 200$ iter	17 s $\rightarrow$ 75 s de entrenamiento
ISMCTS Kuhn 2P $50 \rightarrow 400$ sims	0.04 s $\rightarrow$ 0.28 s por partida

Patrones: el costo de MiniMax sin poda **explota con la profundidad**, sobre todo en juegos grandes (Nocca-Nocca). CFR concentra el costo en el **entrenamiento offline** (una sola vez) y luego juega instantáneamente; en Leduc ese entrenamiento ya es caro. MCTS/ISMCTS pagan el costo **en cada jugada**, proporcional a las simulaciones.

9.2 Performance y equilibrios conocidos

- Información perfecta.** MiniMax y MCTS dominan a Random y entre sí empatan en Tic-Tac-Toe (juego resuelto). En Nocca-Nocca la heurística es decisiva.
- Información imperfecta.** CFR **converge al Nash conocido** en Kuhn 2P ($-1/18$ y farol $J \approx 1/3$). En juegos sin Nash único (Kuhn 3P) o muy grandes (Leduc) no hay un óptimo único alcanzable con pocas iteraciones, pero CFR entrena de forma estable y explota a rivales débiles.

9.3 Resumen por algoritmo

Eje	MiniMax	MCTS	CFR	ISMCTS
Información	Perfecta	Perfecta	Imperfecta	Imperfecta
Entrenamiento previo	No	No	Sí (offline)	No
Garantía teórica	Óptimo (full depth)	Asintótica	Nash (2J suma cero)	Asintótica
Necesita heurística	Sí (depth limitada)	No	No	No

Eje	MiniMax	MCTS	CFR	ISMCTS
Costo dominante	Profundidad	Nº simulaciones	Iteraciones / info sets	Nº simulaciones

10. Conclusiones globales

1. **El eje es el "trade-off" entre cómputo y calidad de juego.** Cada algoritmo lo paga de forma distinta: MiniMax en profundidad, MCTS/ISMCTS en simulaciones por jugada, CFR en iteraciones de entrenamiento offline.
2. **Las funciones de evaluación son críticas en juegos grandes.** En Nocco-Nocco el progreso hacia la meta es la señal dominante; la movilidad sola es la heurística más débil.
3. **CFR es el método de referencia para información imperfecta** cuando hay tiempo de entrenamiento: converge al Nash en Kuhn 2P y es estable en 3P. ISMCTS es una alternativa potente sin entrenamiento, pero más cara por jugada y menos precisa.
4. **La posición influye en todos los juegos.** Abrir es ventajoso en los de tablero y desventajoso en Kuhn; en Kuhn 3P el orden de juego reparte ventajas de forma no trivial.
5. **Honestidad sobre la varianza.** Varios experimentos usan pocas partidas por costo; las conclusiones se apoyan en tendencias y en el contraste con la teoría, no en decimales puntuales.

11. Uso de Inteligencia Artificial Generativa

Siguiendo las pautas de la cátedra, declaramos el uso de IA generativa:

- **Verificación:** todo el contenido generado con IA fue revisado y verificado por el equipo.
- **Herramientas utilizadas:** asistentes de programación basados en LLM (*Claude*).

12. Repositorio

Código y experimentos: <https://github.com/joacoguerrae/obligatorio2-sma-guerra-sanés>